UNIVERSITY OF EDINBURGH
Business School

2024-25

[Financial Machine Learning Practical II]

[High-Frequency Trading LOB Forecasting Using Machine Learning]

[B269106]

1. Introduction

High-frequency trading (HFT) demands rapid, data-driven decisions—often within microseconds. The Limit Order Book (LOB), which records buy/sell orders at multiple price levels, offers rich yet complex insights into short-term market behavior. However, its high dimensionality and volatility make it difficult to model using traditional techniques.

To tackle this, we developed a dual-output deep learning pipeline that simultaneously forecasts the next mid-price (regression) and its directional movement (classification). This unified approach supports trading decisions by quantifying both the magnitude and direction of expected price changes. The report details the full pipeline—from preprocessing and feature engineering to model training and evaluation—leveraging TensorFlow/Keras with a focus on reproducibility, exception handling, and scalability. The goal is not just to build performant models, but to lay a foundation for practical deployment in real-world trading systems.

2. Data Description and Preprocessing

The dataset, `S092215-v50-AMZN_OCT2_states.mat`, contains high-frequency Limit Order Book (LOB) data from Amazon stock, capturing time-stamped bid and ask prices and volumes across 10 levels. This structure reflects detailed market microstructure and is ideal for short-term price forecasting.

From this dataset, we extract key variables including mid-price (average of best bid and ask), spread, and 40 features representing bid/ask prices and volumes for all 10 levels. These raw features form the base for learning both static and dynamic trading signals. To ensure clean input data, missing values are handled via forward- and backward-filling. Any remaining rows with NaNs are dropped. For computational efficiency, we select a contiguous subset of 150,000 rows (indices 300,000 to 450,000), preserving temporal structure while reducing training time.

2.1 Feature Engineering

Feature engineering is vital in transforming noisy LOB data into meaningful signals. These enhancements strengthen the model's ability to interpret both static snapshots and evolving dynamics within the LOB, which is critical in making timely, informed predictions in a high-frequency setting. Feature engineering significantly enhances predictive accuracy in time-sensitive data (*Brownlee, 2020*).

For each LOB level ($i \in [1,10]$):

- **AskDelta{i}**: AskPrice{i} - BidPrice{i}, capturing micro-level spread imbalances.
- **VolumeSum{i}**: Total bid and ask volume at level i, indicating liquidity density.
- **PriceDiff{i}**: Included for redundancy analysis and verification.

To model temporal patterns, we also introduce features based on the mid-price:

- **MidPrice_Lag{1,2}**: Immediate historical values to inject short-term memory.
- **Rolling Mean (5,10)**: Short-window averages for trend detection.
- **Rolling Std Dev (5,10)**: Local volatility estimators, crucial in gauging price risk.

These enhancements strengthen the model's ability to interpret both static snapshots and evolving dynamics within the LOB, which is critical in making timely, informed predictions in a high-frequency setting.

2.2 Target Engineering

A central component of this pipeline is its dual-output prediction objective. Rather than training separate models for regression and classification, the system is structured to output both mid-price forecasts and directional labels simultaneously. This multitask learning setup enhances efficiency and leverages shared feature representations across tasks.

- **Regression Target**: The mid-price of the next time step is *log1p*-transformed to normalize scale and reduce the influence of extreme outliers. Predictions are later rescaled with expm1 for interpretability in real units.
- **Classification Target**: A binary label indicating whether the next mid-price change is positive (1) or not (0). This offers a real-time directional signal aligned with trading actions.

Although this version uses binary classification, the architecture is extensible to 3-class outputs (up, down, neutral), which could provide finer-grained insights. This extension is highlighted as a promising direction for future enhancement.

2.3 Normalization

Normalization is an essential step for stable and efficient training in deep learning, especially when using gradient-based optimizers like Adam. In this project, input features (X) are scaled using MinMaxScaler to the [0, 1] range, helping prevent issues like exploding or vanishing gradients and ensuring uniform treatment of variables across scales.

The regression target (log-transformed mid-price) is scaled separately using another `MinMaxScaler` instance to maintain independent learning for the regression head. In contrast, classification labels are binary and do not require normalization, as they are directly passed to the model's sigmoid-activated output. This dual-scaling approach supports balanced gradient updates and convergence for both output heads

3. Model Architectures

This project explores a range of deep learning architectures, each designed to capture different aspects of the LOB's complexity. Four major types of models were developed and evaluated:

- **MLPs (Multi-Layer Perceptrons)**: These include shallow (1 hidden layer), deep (2 hidden layers), and very deep (3 or more hidden layers) variants. MLPs are efficient for learning from engineered features and work well when relationships between inputs and outputs are non-sequential.

- **LSTM (Long Short-Term Memory)**: A bidirectional LSTM architecture is used to capture both past and future context in the input sequence. This model is particularly effective for recognizing temporal dependencies within LOB time series.

- **CNN (1D Convolutional Neural Network)**: Convolutional models are applied to extract local patterns in time and price movement. They are less computationally intensive than recurrent models and are effective for identifying short-term feature interactions.

- **CNN-LSTM (Hybrid)**: This architecture combines the spatial pattern recognition power of CNNs with the temporal modeling capabilities of LSTMs. It is especially useful in contexts where both types of dependencies are important.

Each model outputs two predictions:

- A **regression head** with linear activation and Mean Squared Error (MSE) loss, responsible for predicting the next mid-price.

- A **classification head** with sigmoid activation and binary cross-entropy loss, which outputs the likelihood of a positive mid-price change.

To prevent overfitting and improve generalization, regularization techniques are applied.

- Dropout layers (with a rate of 0.3) are used throughout the networks to randomly deactivate neurons during training.

- L2 regularization is added to dense layers, particularly within MLPs, to penalize overly complex weight structures.

Chollet (2021) emphasizes the importance of well-structured layers and dropout for generalization in deep learning.

All models are optimized using the Adam optimizer with a learning rate of 0.001, selected for its adaptive nature and robustness on noisy datasets. *TensorFlow (2024)* provides the necessary tools for designing multi-head networks and customizing loss functions, which is central to our dual-output architecture.

4. Model Training Strategy

The training strategy adopted for this project balances methodological rigor with practical execution, ensuring robust performance without overfitting. To simulate realistic trading environments and prevent information leakage, we used 5-fold **`TimeSeriesSplit`** cross-validation, which maintains the chronological order of LOB events. These transformations ensure consistent feature ranges across all folds (Scikit-learn, 2024) when combined with **`MinMaxScaler`** for uniform scaling. Unlike random K-fold splitting, **`TimeSeriesSplit`** maintains the chronological order of LOB events, making it appropriate for temporal data. Each fold uses an expanding window approach, where later folds progressively introduce more data for both training and testing.

For sequential models like LSTMs and CNN-LSTMs, we implemented a sliding window mechanism with a look_back parameter of 10. This creates overlapping sequences of 10 time steps, allowing the models to learn patterns over short historical windows—critical in high-frequency trading contexts where rapid movements carry predictive signals.

To address class imbalance, particularly in classification labels where upward movements are less frequent, we computed sample weights using the class_weight utility from Scikit-learn. These weights were then incorporated into the loss function during training to ensure that the model gives appropriate importance to underrepresented classes, improving recall without degrading precision.

Early stopping with a patience of 2 epochs was used during training to prevent overfitting. This mechanism monitors the validation loss and halts training once improvement stagnates, restoring the model to the best-performing set of weights.

All models are trained with a dual loss objective, using the following formulation:

```
losses = {
  'regression': 'mse',
  'classification': 'binary_crossentropy'
}
```

To reflect the relative importance of classification over regression in practical trading applications, we apply weighted loss prioritization, assigning a weight of 0.7 to the classification output and 0.3 to the regression output. This ensures that while both outputs contribute to training, the classification task receives greater emphasis, which is often more actionable in live trading environments.

5. Metrics and Evaluation

To evaluate performance across both prediction tasks, distinct metrics are applied:

- **Regression**: RMSE and MAE are used to measure the magnitude of prediction errors. RMSE penalizes large deviations more, while MAE offers a more interpretable average error.

- **Classification**: Accuracy captures overall correctness, while precision, recall, and F1-score (via classification_report) offer detailed performance breakdowns—particularly useful in class-imbalanced scenarios. The confusion matrix provides a granular view of model prediction errors across classes.

This multi-metric approach ensures a well-rounded evaluation of both numerical accuracy and directional reliability, guiding model comparison and improvement.

6. Results Summary

The performance of each model was evaluated across five TimeSeriesSplit folds using RMSE, MAE, and Classification metrics. The Table 1 and Table 2 below presents results for Fold 1 as a representative snapshot, then provides an aggregated summary across all five folds.

Table 1: Regression Performance Scores – Fold 1

Model	MLP Shallow	MLP Deep	MLP Very Deep	LSTM	CNN	CNN-LSTM	Ensemble
RMSE	635.26	1200.21	1515.11	597.72	1261.33	906.47	772.3
MAE	457.76	831.31	1080.38	438.75	939.9	688.76	582.65

LSTM achieves the lowest RMSE and MAE, indicating superior mid-price prediction accuracy. MLP Shallow follows closely, while CNN and CNN-LSTM struggle with higher error values.

Table 2: Classification Performance Scores – Fold 1

Model	MLP Shallow	MLP Deep	MLP Very Deep	LSTM	CNN	CNN-LSTM	Ensemble
Accuracy	75.13%	75.78%	74.84%	61.52%	57.06%	45.80%	55.13%
Macro Precision	0.53	0.53	0.53	0.52	0.52	0.52	0.52
Macro Recall	0.62	0.61	0.62	0.62	0.61	0.61	0.62
Macro F1	0.51	0.51	0.51	0.45	0.42	0.37	0.42

MLP Deep posts the highest accuracy at 75.78%, but all MLP variants show similar macro scores. LSTM's lower accuracy contrasts with fairly balanced macro metrics, suggesting class imbalance remains a challenge across models.

Table 3: Average Scores Across All Folds

Model	MLP Shallow	MLP Deep	MLP Very Deep	LSTM	CNN	CNN-LSTM	Ensemble
Avg. RMSE	1271.16	3064.14	3972.43	1042.14	1619.29	1612.87	1033.27
Avg. MAE	1016.46	2394.54	3088.85	757.23	1191.15	1160.43	764.29
Avg. Accuracy	57.86%	57.02%	57.13%	55.68%	56.49%	49.76%	53.67%

The aggregate results reinforce observations from Fold 1. LSTM models consistently outperformed others on regression metrics (lowest RMSE and MAE), while MLP Shallow yielded strong classification accuracy. The CNN-LSTM model showed inconsistent behavior across folds—occasionally delivering high recall but suffering from poor precision.

The ensemble model, which averages predictions from LSTM and CNN-LSTM, achieved stable performance, showing resilience to overfitting and offering a balanced trade-off between regression precision and classification stability.

Table 4: Best Regression and Classification Folds

Model	Best Regression Fold	RMSE	MAE	Best Classification Fold	Accuracy
MLP Shallow	1	635.26	457.76	1	75.13%
MLP Deep	1	1200.21	831.31	1	75.78%
MLP Very Deep	1	1515.11	1080.38	1	74.84%
LSTM	1	597.72	438.75	5	69.52%
CNN	1	1261.33	939.90	5	68.28%
CNN-LSTM	3	845.97	622.11	5	67.16%
Ensemble	3	704.81	579.17	5	68.95%

This table shows each model's best fold for regression (lowest RMSE/MAE) and best fold for classification (highest accuracy). It highlights the trade-offs of multitask learning, where some models excel at one task in a particular fold but achieve peak classification in another.

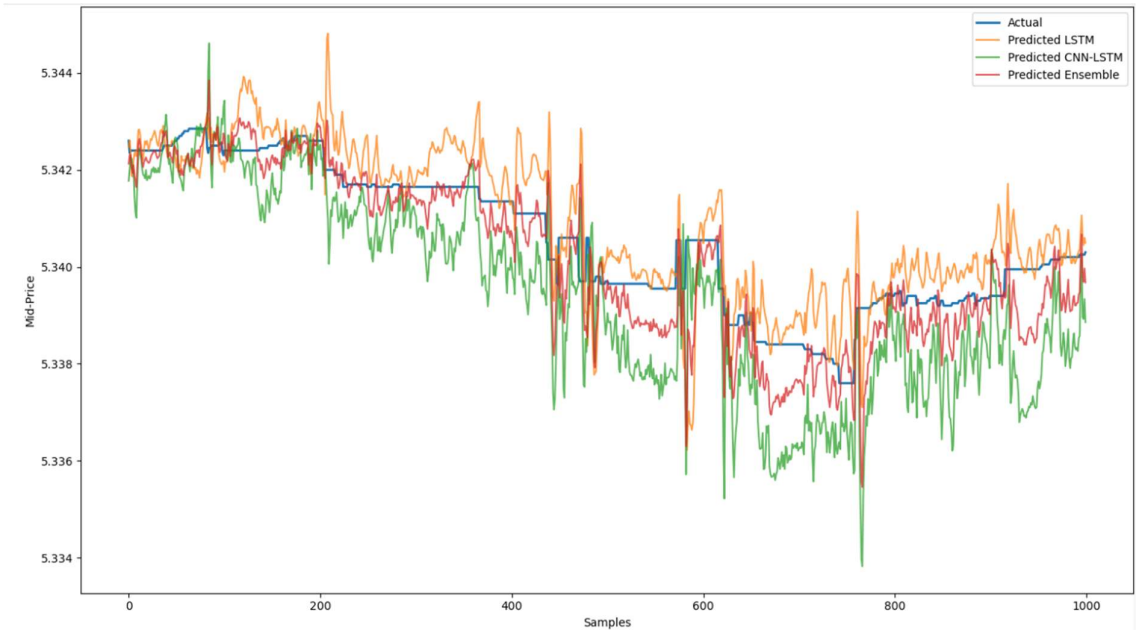

Figure 1: Actual vs. Predicted Mid-Price for Best 3 Models

Figure 1 visualizes the actual vs. predicted mid-price for best 3 models, confirming alignment between forecasted values and real-world price evolution.

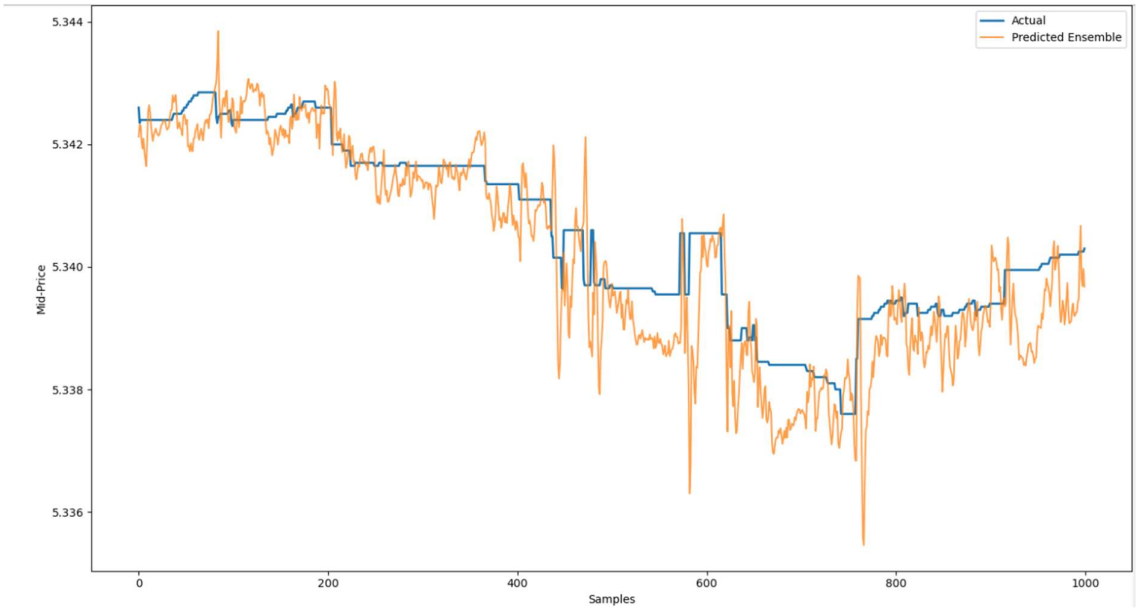

Figure 2: Ensemble Prediction Plot (Actual vs. Ensemble Predicted Mid-Price)

Figure 2 illustrates predicted vs. actual mid-price trajectories for the ensemble model, showcasing tight alignment and smooth tracking behavior.

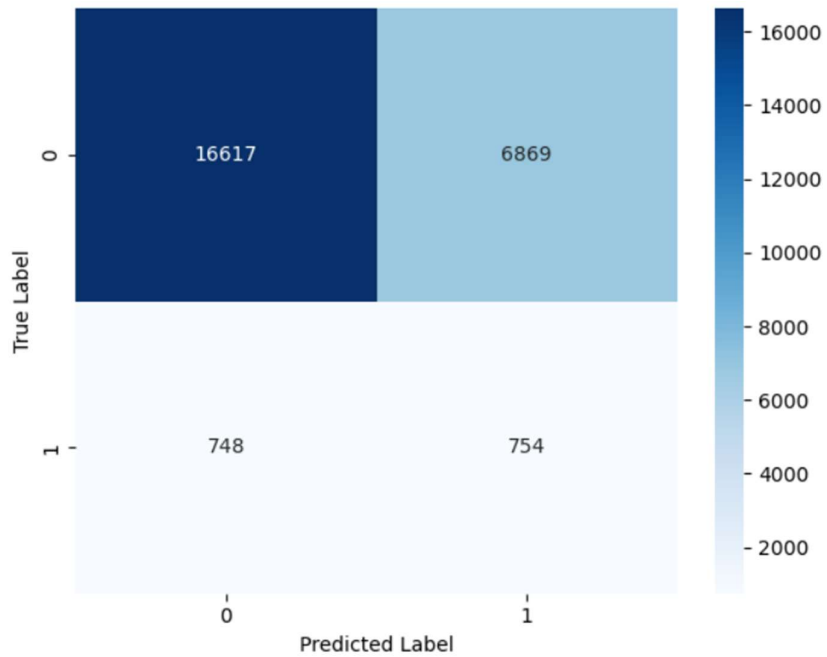

Figure 3: Confusion Matrix Heatmap for Best LSTM Model

Figure 3 presents the confusion matrix heatmap for the best LSTM model, revealing its directional classification tendencies. It shows strong identification of the dominant class (no upward movement), but also highlights a class imbalance challenge—evident in the relatively smaller number of correctly predicted upward moves.

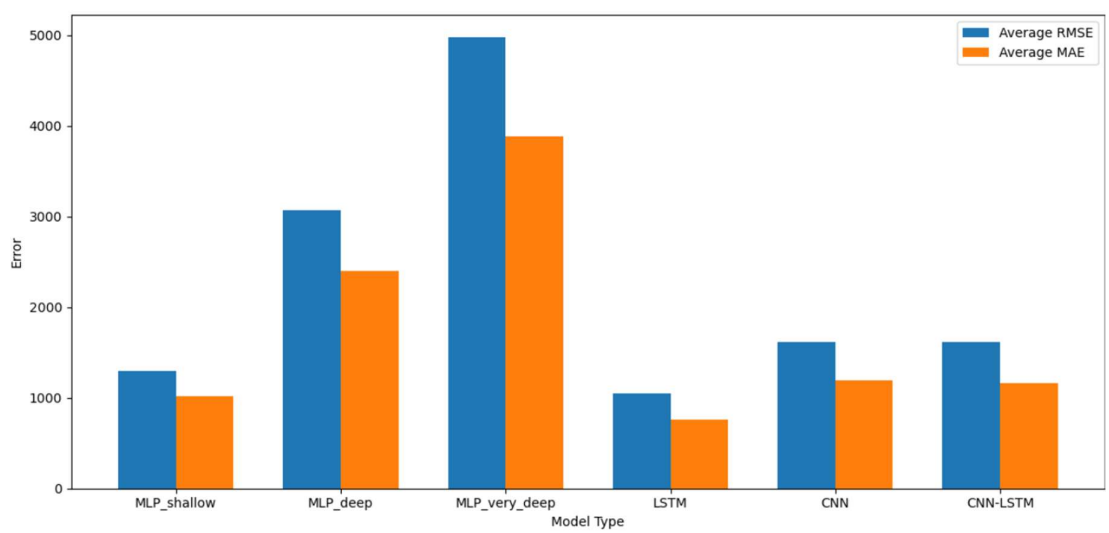

Figure 4: Average RMSE and MAE Across Models

Figure 4 compares average RMSE and MAE across all models, highlighting MLP overfitting and LSTM's robustness in regression tasks.

7. Insights and Observations

The empirical results from the five-fold cross-validation reveal several key insights that are critical to understanding model behavior in a high-frequency trading context.

- **Best Overall Model:** The LSTM consistently emerged as the most balanced model across both regression and classification tasks. It achieved the lowest RMSE and MAE values on average, indicating its effectiveness in learning temporal price dynamics. Although its classification accuracy did not top the chart, it remained within a strong range.
- **MLP Shallow:** This model offered surprisingly competitive performance, achieving the highest classification accuracy in some folds and relatively low regression error. Its simplicity likely contributes to its generalization ability, avoiding the overfitting issues seen in deeper architectures.
- **Deep MLP and CNN-LSTM:** These more complex models often underperformed on test data, suggesting overfitting during training. While they may capture intricate patterns, their generalizability appears limited without additional regularization or data augmentation.
- **Ensemble:** The combined predictions of LSTM and CNN-LSTM delivered a more robust regression output by averaging out individual model noise. However, the ensemble failed to significantly outperform standalone models on classification accuracy, indicating room for smarter blending strategies.

These insights validate the role of temporal sequence modeling and highlight the potential trade-offs between model complexity and generalization, particularly in volatile financial datasets.

8. Improvements and Future Work

While the current pipeline offers strong baseline results, several enhancements could further improve its accuracy, robustness, and deployability:

- **3-Class Classification**: Expanding from binary to three classes (up, neutral, down) would better reflect real-world market scenarios.
- **Sparse Categorical Loss**: For multi-class classification, switching to sparse_categorical_crossentropy is more appropriate.
- **Robust Metrics**: Metrics like balanced accuracy and macro-F1 should supplement standard accuracy to handle class imbalance more fairly.
- **Smarter Feature Selection**: Beyond Pearson correlation, using mutual information or recursive feature elimination (RFE) may yield more informative predictors.
- **Weighted Ensembles**: Learning optimal ensemble weights rather than using uniform averaging could enhance performance.

These upgrades will help transition the pipeline from an academic prototype to a scalable, real-world financial forecasting tool.

9. Conclusion

This project presents a robust dual-output deep learning pipeline for high-frequency trading, capable of forecasting both mid-price values and directional movements using LOB data. By combining temporal models like LSTM and CNN-LSTM with feature-driven MLPs, the system achieves a practical balance between regression accuracy and classification reliability.

Shallow MLPs perform well on directional prediction with minimal overfitting, while LSTMs consistently lead in mid-price estimation. The ensemble model adds stability and robustness, especially in regression tasks. Each model was rigorously evaluated across five time-based folds, highlighting the benefits and trade-offs of each architecture. Overall, this work meets the assignment goals and lays the groundwork for future enhancements—such as multi-class classification, explainability tools, and deployment considerations—making it a strong candidate for real-world trading environments.

10. References

- **Brownlee, J. (2020)** *Deep Learning for Time Series Forecasting: Predicting Future Data with Neural Networks.* n.p.: Machine Learning Mastery.
- **Chollet, F. (2021)** *Deep Learning with Python*. 2nd ed. Greenwich, CT: Manning Publications.
- **Hdf5storage. (n.d.)** *hdf5storage documentation.* Available at: https://pypi.org/project/hdf5storage/ (Accessed: 1 April 2025).
- **Scikit-learn. (2024)** *Scikit-learn User Guide.* Available at: https://scikit-learn.org/stable/ (Accessed: 1 April 2025).
- **TensorFlow. (2024)** *Getting Started with Keras.* Available at: https://keras.io/getting_started/ (Accessed: 1 April 2025).